BT-AIML-204 A

Machine Learning Techniques

Unit 3 & Unit 4 — Complete Study Notes with Numericals

Poornima Institute of Engineering & Technology, Panipat

B.Tech CSE (AI/ML) · 4th Semester

Contents at a Glance

UNIT 3 — Supervised Learning | 10 Hours

3.1 Linear Regression

★ 12-Mark Topic — OLS derivation + Gradient Descent + Numerical — high weightage

What is Linear Regression?

Linear Regression is used to predict a **continuous numerical output** (e.g., house price, temperature) based on one or more input features. It assumes a **linear relationship** between inputs and output.

Simple Linear Regression

Equation: y = β■ + β■x where β■ = intercept (bias), β■ = slope (weight), x = input feature, y = predicted output

- β■ = Cov(x,y) / Var(x) = Σ(x■–x■)(y■–■) / Σ(x■–x■)²
- β■ = ■ – β■x■

Multiple Linear Regression

Equation: y = β■ + β■x■ + β■x■ + ... + β■x■ (n features)

Key Assumptions of Linear Regression

Assumption	Description
Linearity	Relationship between X and y is linear
Independence	Observations are independent of each other
Homoscedasticity	Variance of residuals is constant (no pattern in errors)
Normality	Residuals are normally distributed
No Multicollinearity	Features are not highly correlated with each other

Ordinary Least Squares (OLS)

OLS minimises the **Sum of Squared Residuals (SSR)**. A residual is the difference between the actual value and the predicted value.

Cost Function (MSE): J(β) = (1/n) × Σ(y■ – ■■)²

OLS finds β values analytically: **β = (X■X)■¹ X■y**

- Advantage: Exact solution in one step — no iteration needed.
- Disadvantage: Computationally expensive for very large datasets (matrix inversion is O(n³)).

Gradient Descent

An iterative optimisation algorithm used when OLS is too slow (big data). It starts with random β values and updates them step by step in the direction that reduces the cost.

Update Rule: β_new = β_old – α × (∂J/∂β)

where α **(alpha)** = learning rate (controls step size)

- Too small α → very slow convergence.
- Too large α → overshoots minimum, may diverge.

Type	Description	Use When
Batch GD	Uses ALL training data per update. Stable but slow for large data.	Small datasets
Stochastic GD	Uses ONE random sample per update. Fast but noisy.	Very large datasets
Mini-batch GD	Uses a BATCH (e.g., 32 or 64 samples). Best of both worlds.	Most common in practice

Polynomial Regression

When data shows a curved (non-linear) relationship, we add polynomial terms to the model: **y = β■ + β■x + β■x² + β■x³ + ...**

- Still called 'linear regression' because the model is linear in the coefficients (β), not in x.
- Higher degree → more flexible → risk of overfitting.

NUMERICAL: Simple Linear Regression — Find Best Fit Line	
Given	x = [1, 2, 3, 4, 5], y = [2, 4, 5, 4, 5]
Step 1	n = 5, x■ = (1+2+3+4+5)/5 = 15/5 = 3.0, ■ = (2+4+5+4+5)/5 = 20/5 = 4.0
Step 2	Σ(x■–x■)(y■–■) = (1–3)(2–4)+(2–3)(4–4)+(3–3)(5–4)+(4–3)(4–4)+(5–3)(5–4) = (–2)(–2)+(–1)(0)+(0)(1)+(1)(0)+(2)(1) = 4+0+0+0+2 = 6
Step 3	Σ(x■–x■)² = (–2)²+(–1)²+(0)²+(1)²+(2)² = 4+1+0+1+4 = 10
Step 4	β■ = 6/10 = 0.6
Step 5	β■ = ■ – β■x■ = 4.0 – 0.6×3.0 = 4.0 – 1.8 = 2.2
Answer	Best fit line: ■ = 2.2 + 0.6x Prediction for x=6: ■ = 2.2 + 0.6×6 = 5.8

NUMERICAL: MSE Calculation	
Given	Actual y = [3, 5, 2, 8], Predicted ■ = [2.5, 5.0, 4, 8]
Step 1	Residuals = y – ■ = [0.5, 0, –2, 0]
Step 2	Squared residuals = [0.25, 0, 4, 0]
Step 3	MSE = (0.25 + 0 + 4 + 0) / 4 = 4.25 / 4 = 1.0625
Answer	MSE = 1.0625 | RMSE = √1.0625 ≈ 1.031

3.2 Logistic Regression — Binary & Multiclass

★ 12-Mark Topic — Sigmoid function + decision boundary + numerical

Despite the name, Logistic Regression is a **classification** algorithm, not regression. It predicts the **probability** that an input belongs to a class (0 or 1).

Sigmoid Function

The sigmoid (logistic) function converts any real number into a probability between 0 and 1:

$\sigma(z) = 1 / (1 + e^{-z})$ where z = β■ + β■x■ + β■x■ + ...

- If σ(z) ≥ 0.5 → predict class 1

- If σ(z) < 0.5 → predict class 0
- The threshold 0.5 can be adjusted based on the problem.

Cost Function

We cannot use MSE for logistic regression (non-convex). Instead we use **Binary Cross-Entropy Loss**:

J = –(1/n) × Σ [y■ log(■■) + (1–y■) log(1–■■)]

Multiclass Classification

Strategy	Description	Example
One-vs-Rest (OvR)	Train one classifier per class: class k vs all others. Pick class with highest probability.	3 classes → 3 binary classifiers
One-vs-One (OvO)	Train one classifier for each pair of classes. Vote among all classifiers.	3 classes → 3 binary classifiers
Softmax Regression	Extension of logistic regression: outputs probability for each class directly.	Deep learning multiclass

NUMERICAL: Sigmoid Calculation	
Given	z = 2.0
Formula	$\sigma(z) = 1 / (1 + e^{-z}) = 1 / (1 + e^{-2})$
Step 1	$e^{-2} = 0.1353$
Step 2	σ(2) = 1 / (1 + 0.1353) = 1 / 1.1353 = 0.8808
Answer	Probability = 0.88 → Since ≥ 0.5, predict Class 1

NUMERICAL: Cross-Entropy Loss	
Given	Actual y = [1, 0, 1], Predicted probabilities ■ = [0.9, 0.2, 0.7]
Step 1	Loss for each sample: Sample 1: –[1×log(0.9) + 0×log(0.1)] = –log(0.9) = 0.105 Sample 2: –[0×log(0.2) + 1×log(0.8)] = –log(0.8) = 0.223 Sample 3: –[1×log(0.7) + 0×log(0.3)] = –log(0.7) = 0.357
Answer	Average Loss = (0.105 + 0.223 + 0.357) / 3 = 0.685 / 3 ≈ 0.228

3.3 Model Evaluation — Confusion Matrix & Cross-Validation

★ 12-Mark Topic — Confusion matrix + all metrics formula + numerical

Confusion Matrix

A confusion matrix summarises how well a classification model performed on a dataset. Rows = Actual, Columns = Predicted.

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP (True Positive) Correctly predicted Positive	FN (False Negative) Actually Positive, predicted Negative (Type II Error)
Actual: Negative	FP (False Positive) Actually Negative, predicted Positive (Type I Error)	TN (True Negative) Correctly predicted Negative

Performance Metrics from Confusion Matrix

Metric	Formula	Meaning	When to Use
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correct predictions	Balanced classes
Precision	TP/(TP+FP)	Of all predicted positives, how many were actually positive?	When FP is costly (spam filter)
Recall	TP/(TP+FN)	Of all actual positives, how many did we catch?	When FN is costly (cancer detection)
F1-Score	2×(P×R)/(P+R)	Harmonic mean of Precision and Recall	Imbalanced datasets
Specificity	TN/(TN+FP)	True Negative Rate	Medical screening

NUMERICAL: Confusion Matrix — All Metrics

Given	Confusion Matrix: TP = 50, FP = 10, FN = 5, TN = 35
Total	n = 50+10+5+35 = 100
Accuracy	(50+35)/100 = 85/100 = 0.85 = 85%
Precision	50/(50+10) = 50/60 = 0.833 = 83.3%
Recall	50/(50+5) = 50/55 = 0.909 = 90.9%
F1-Score	2×(0.833×0.909)/(0.833+0.909) = 2×0.757/1.742 = 1.514/1.742 ≈ 0.869
Specificity	35/(35+10) = 35/45 = 0.778 = 77.8%

Cross-Validation

Cross-validation is a technique to evaluate model performance more reliably by testing on multiple different splits of data.

- **k-Fold Cross-Validation:** Split data into k equal parts (folds). Train on k–1 folds, test on 1 fold. Repeat k times. Average all k scores.
- **Leave-One-Out (LOOCV):** k = n (one sample as test each time). Very thorough but computationally expensive.
- **Stratified k-Fold:** Ensures each fold has the same class distribution. Best for imbalanced datasets.

Remember: *Cross-validation final score = mean of all fold scores. It is more reliable than a single train-test split.*

3.4 Decision Trees

★ 12-Mark Topic — Entropy + Information Gain + build tree numerical

A Decision Tree is a tree-shaped model that makes decisions by splitting data based on feature values. It is easy to understand and visualise — works like a flowchart of yes/no questions.

Key Concepts

Term	Meaning
Root Node	The very first split — the most important feature
Internal Node	A decision/split point on a feature
Leaf Node	Final output — the class label or predicted value

Term	Meaning
Branch	The path from one node to another based on a condition
Depth	Number of levels in the tree — more depth = more complex

Splitting Criteria — How to Pick the Best Feature?

1. Entropy (used in ID3, C4.5 algorithms):

Entropy measures **impurity** (disorder) in a node.

Entropy H(S) = –Σ p■ × log■(p■)

- H = 0 → Pure node (all same class) — best case
- H = 1 → Maximum impurity (50-50 split) — worst case

2. Information Gain (IG):

IG = Entropy(parent) – Σ [|S■|/|S| × Entropy(S■)]

- We pick the feature with the HIGHEST Information Gain to split on.

3. Gini Impurity (used in CART):

Gini(S) = 1 – Σ p■²

- Gini = 0 → Perfect purity
- Gini = 0.5 → Maximum impurity (binary case)

NUMERICAL: Entropy and Information Gain	
Problem	Dataset S has 14 samples: 9 Yes, 5 No (play tennis). Feature 'Outlook' splits into: Sunny: 5 samples (2 Yes, 3 No) Overcast: 4 samples (4 Yes, 0 No) Rain: 5 samples (3 Yes, 2 No)
Step 1 — Entropy of S	H(S) = –(9/14)log■(9/14) – (5/14)log■(5/14) = –(0.643)×(–0.637) – (0.357)×(–1.485) = 0.410 + 0.530 = 0.940
Step 2 — Entropy of each branch	H(Sunny) = –(2/5)log■(2/5) – (3/5)log■(3/5) = 0.529+0.442 = 0.971 H(Overcast) = –(4/4)log■(4/4) = 0 (pure node!) H(Rain) = –(3/5)log■(3/5) – (2/5)log■(2/5) = 0.442+0.529 = 0.971
Step 3 — Weighted avg	Weighted = (5/14)×0.971 + (4/14)×0 + (5/14)×0.971 = 0.347 + 0 + 0.347 = 0.693
Answer	IG(Outlook) = H(S) – Weighted = 0.940 – 0.693 = 0.247 This means Outlook reduces uncertainty by 0.247 bits — choose it as the root if it has the highest IG.

NUMERICAL: Gini Impurity	
Given	Node has 10 samples: 6 Class A, 4 Class B
Step 1	p(A) = 6/10 = 0.6, p(B) = 4/10 = 0.4
Step 2	Gini = 1 – (p(A)² + p(B)²) = 1 – (0.36 + 0.16) = 1 – 0.52 = 0.48
Answer	Gini = 0.48 (moderately impure). A pure node would give Gini = 0.

3.5 K-Nearest Neighbours (KNN)

❖ **6-Mark Topic — Algorithm + distance + numerical**

KNN is a simple, non-parametric, lazy learning algorithm. It classifies a new data point based on the majority class of its K nearest neighbours in the training set. No explicit training — it memorises all data.

Algorithm Steps

- Step 1: Choose K (number of neighbours).
- Step 2: Calculate distance from the new point to all training points.
- Step 3: Sort distances and pick the K smallest (K nearest neighbours).
- Step 4: Majority vote → assign the most common class among K neighbours (for regression: take mean).

Distance Metrics

Metric	Formula	Use
Euclidean	$\sqrt{\Sigma(x_i - y_i)^2}$	Most common — works well for continuous data
Manhattan	$\Sigma\|x_i - y_i\|$	Grid-like paths, robust to outliers
Minkowski	$(\Sigma\|x_i - y_i\|^p)^{1/p}$	Generalisation: p=1 Manhattan, p=2 Euclidean

NUMERICAL: KNN Classification (K=3)	
Given	Training data (x1, x2, Label): A=(1,2,+), B=(2,3,+), C=(3,1,+), D=(6,5,–), E=(7,8,–) | New point P=(3,3)
Distances from P=(3,3)	d(P,A) = $\sqrt{((3-1)^2+(3-2)^2)} = \sqrt{(4+1)} = \sqrt{5} \approx 2.24$ d(P,B) = $\sqrt{((3-2)^2+(3-3)^2)} = \sqrt{(1+0)} = \sqrt{1} = 1.00$ d(P,C) = $\sqrt{((3-3)^2+(3-1)^2)} = \sqrt{(0+4)} = \sqrt{4} = 2.00$ d(P,D) = $\sqrt{((3-6)^2+(3-5)^2)} = \sqrt{(9+4)} = \sqrt{13} \approx 3.61$ d(P,E) = $\sqrt{((3-7)^2+(3-8)^2)} = \sqrt{(16+25)} = \sqrt{41} \approx 6.40$
K=3 nearest	B (1.00), C (2.00), A (2.24) → all are class +
Answer	Majority class = (+), so New point P is classified as POSITIVE (+)

3.6 Naive Bayes Classifier — Bayes Theorem

★ 12-Mark Topic — Bayes theorem + posterior probability numerical

Bayes Theorem

P(A|B) = P(B|A) × P(A) / P(B)

In ML context: **P(Class|Features) = P(Features|Class) × P(Class) / P(Features)**

Term	Name	Meaning
P(Class|Features)	Posterior	Probability of class given the features — what we want
P(Features|Class)	Likelihood	Probability of seeing these features given the class
P(Class)	Prior	Probability of the class before seeing any features
P(Features)	Evidence	Normalising constant — same for all classes, can be ignored

Why 'Naive'?

It assumes all features are **conditionally independent** given the class — a strong (often unrealistic) assumption, hence 'naive'. Despite this, it works surprisingly well in practice for text classification.

Variant	Distribution Assumed	Use Case
Gaussian NB	Features follow normal distribution	Continuous numerical features
Multinomial NB	Features are counts/frequencies	Text classification (word counts)
Bernoulli NB	Features are binary (0/1)	Binary feature data

NUMERICAL: Naive Bayes — Email Spam Classification	
Given	Training: 60 Ham emails, 40 Spam emails (100 total) Word 'lottery' appears in 5 Ham and 35 Spam emails Classify new email that contains word 'lottery'
Priors	P(Ham) = 60/100 = 0.60 P(Spam) = 40/100 = 0.40
Likelihoods	P('lottery'|Ham) = 5/60 = 0.0833 P('lottery'|Spam) = 35/40 = 0.875
Posteriors (un normalised)	P(Ham|'lottery') ∝ 0.0833 × 0.60 = 0.050 P(Spam|'lottery') ∝ 0.875 × 0.40 = 0.350
Answer	P(Spam) score 0.350 >> P(Ham) score 0.050 → Classify as SPAM. Normalised: P(Spam|lottery) = 0.350/(0.350+0.050) = 0.875 = 87.5%

3.7 Support Vector Machines (SVM)

★ 12-Mark Topic — Hyperplane + margin + kernel trick explanation

SVM finds the **optimal hyperplane** that best separates two classes with the **maximum margin**. It focuses on the most difficult-to-classify points called **support vectors**.

Key Concepts

Term	Definition
Hyperplane	Decision boundary separating the classes. For 2D: a line; for 3D: a plane.
Support Vectors	The data points closest to the hyperplane — they define and support the margin.
Margin	Distance between the two parallel support hyperplanes. SVM maximises this.
Hard Margin SVM	Requires all points to be correctly classified — only for linearly separable data.
Soft Margin SVM	Allows some misclassifications. Controlled by parameter C.

The C Parameter (Regularisation)

- Large C → Small margin, fewer misclassifications on training data (risk of overfitting).
- Small C → Large margin, allows more misclassifications (better generalisation).

Kernel Trick

When data is **not linearly separable**, the kernel trick maps data into a higher-dimensional space where a linear separator exists — without explicitly computing the transformation (very efficient).

Kernel	Formula	Use Case
Linear	K(x,y) = x■y	Linearly separable data, high-dimensional (text)
Polynomial	K(x,y) = (γx■y + r)■	Non-linear boundaries, image recognition
RBF/Gaussian	K(x,y) = exp(–γ||x–y||²)	Most popular, works well for most problems
Sigmoid	K(x,y) = tanh(γx■y + r)	Used in neural networks, NLP tasks

NUMERICAL: SVM — Verify if Point is Support Vector	
Given	SVM hyperplane: 2x■ + 3x■ – 6 = 0 Points: A=(0,2,+), B=(3,0,+), C=(1,1,–), D=(2,2,+)
Step	Substitute each point into 2x■ + 3x■ – 6: A: 2(0)+3(2)–6 = 0 → ON the margin boundary (support vector!) B: 2(3)+3(0)–6 = 0 → ON the margin boundary (support vector!) C: 2(1)+3(1)–6 = –1 → Inside negative region (correct) D: 2(2)+3(2)–6 = +4 → Inside positive region (correct)
Answer	Points A and B are Support Vectors — they lie exactly on the margin boundaries.

UNIT 4 — Unsupervised Learning | 8 Hours

Unsupervised learning finds hidden patterns and structures in **unlabelled data**. There are no predefined outputs — the model discovers the structure on its own.

4.1 K-Means Clustering

★ 12-Mark Topic — Algorithm steps + convergence numerical — guaranteed exam Q

K-Means divides n data points into **K clusters**, where each point belongs to the cluster with the **nearest centroid**. It minimises the within-cluster sum of squared distances (WCSS/Inertia).

Algorithm Steps

- **Step 1 — Initialisation:** Randomly pick K data points as initial centroids.
- **Step 2 — Assignment:** Assign each point to the nearest centroid (Euclidean distance).
- **Step 3 — Update:** Recalculate centroids as the mean of all points in each cluster.
- **Step 4 — Repeat:** Go back to Step 2. Stop when centroids stop moving (convergence) or max iterations reached.

Choosing K — The Elbow Method

Run K-Means for K = 1, 2, 3, ... and plot WCSS. The point where the curve bends like an 'elbow' is the optimal K — adding more clusters beyond this gives diminishing returns.

Advantage	Disadvantage
Simple and fast	Must specify K in advance
Scales to large datasets	Sensitive to initial centroid placement (use K-Means++ to fix)
Easy to interpret	Assumes spherical, equal-sized clusters
Guaranteed to converge	Sensitive to outliers — they shift centroids

NUMERICAL: K-Means Iteration (K=2)	
Given	Points: A(1,1), B(1,2), C(2,1), D(5,4), E(5,5), F(6,4) Initial centroids: C1=(1,1), C2=(5,4)
Iteration 1 — Assignment	Distances to C1=(1,1) and C2=(5,4): A(1,1): d(C1)=0.0, d(C2)=5.00 → Cluster 1 B(1,2): d(C1)=1.0, d(C2)=4.24 → Cluster 1 C(2,1): d(C1)=1.0, d(C2)=3.61 → Cluster 1 D(5,4): d(C1)=5.00, d(C2)=0.0 → Cluster 2 E(5,5): d(C1)=5.66, d(C2)=1.0 → Cluster 2 F(6,4): d(C1)=5.83, d(C2)=1.0 → Cluster 2
Iteration 1 — Update Centroids	Cluster 1 = {A,B,C}: new C1 = ((1+1+2)/3, (1+2+1)/3) = (1.33, 1.33) Cluster 2 = {D,E,F}: new C2 = ((5+5+6)/3, (4+5+4)/3) = (5.33, 4.33)
Iteration 2 — Re-assign (verify no change)	All points still closest to same centroids — algorithm converges!
Answer	Final Clusters: Cluster 1: A(1,1), B(1,2), C(2,1) → Centroid (1.33, 1.33) Cluster 2: D(5,4), E(5,5), F(6,4) → Centroid (5.33, 4.33)

4.2 Hierarchical Clustering

❖ 6-Mark Topic — Agglomerative steps + dendrogram

Hierarchical clustering builds a **tree-like structure (dendrogram)** showing how data points are grouped. No need to specify K in advance.

Type	Direction	Process
Agglomerative (Bottom-Up)	Start with each point as its own cluster. Merge closest pairs step by step. Most common.	n clusters → 1 cluster
Divisive (Top-Down)	Start with one big cluster. Split recursively. Less common.	1 cluster → n clusters

Linkage Methods — How to Measure Distance Between Clusters?

Linkage	Distance Measured	Characteristics
Single Linkage	Minimum distance between any two points in the clusters	Sensitive to outliers, creates chain-like clusters
Complete Linkage	Maximum distance between any two points	Creates compact, spherical clusters
Average Linkage	Average distance between all pairs of points	Compromise — works well in practice
Ward's Method	Minimises increase in within-cluster variance	Most popular — creates well-separated clusters

NUMERICAL: Agglomerative Clustering Step-by-Step	
Given	5 points with distance matrix: A B C D E A [0 2 6 10 9] B [2 0 5 9 8] C [6 5 0 4 5] D [10 9 4 0 3] E [9 8 5 3 0]
Step 1	Smallest distance = 2 (A,B) → Merge A and B → Clusters: {A,B}, C, D, E
Step 2	Recalculate distances. Smallest = 3 (D,E) → Merge D and E → Clusters: {A,B}, C, {D,E}
Step 3	Smallest remaining = 4 ({D,E},C) → Merge → Clusters: {A,B}, {C,D,E}
Step 4	Merge remaining two clusters → Final: {A,B,C,D,E}
Answer	Order of merges: (A,B) → (D,E) → (C,D,E) → All. Cut dendrogram at height 4 to get 2 clusters: {A,B} and {C,D,E}.

4.3 DBSCAN — Density-Based Spatial Clustering

❖ 6-Mark Topic — Core/Border/Noise points + advantage over K-Means

DBSCAN groups points that are closely packed together and marks low-density regions as outliers (noise). Unlike K-Means, it can find **arbitrarily shaped clusters** and automatically detect **outliers**.

Two Key Parameters

- ε **(epsilon):** The radius within which we look for neighbouring points.
- **MinPts:** Minimum number of points (including itself) required within ε to be a core point.

Three Types of Points

Point Type	Definition	Role
Core Point	Has at least MinPts points within radius ε	Centre of a cluster
Border Point	Within ε of a core point but has fewer than MinPts neighbours	On the edge of a cluster
Noise Point	Not within ε of any core point	Outlier — no cluster

DBSCAN Algorithm

- Step 1: Pick an unvisited point. Mark it as visited.
- Step 2: Find all points within ε (its neighbourhood).
- Step 3: If neighbourhood size ≥ MinPts → it's a Core Point. Start a new cluster.
- Step 4: Expand cluster by recursively adding all density-reachable points.
- Step 5: If neighbourhood size < MinPts → mark as Noise (may later be assigned as Border).
- Step 6: Repeat until all points are visited.

Feature	K-Means	DBSCAN
K needed?	Yes — must specify K	No — discovers automatically
Cluster Shape	Spherical only	Any shape (irregular, non-convex)
Outlier Handling	Outliers distort centroids	Explicitly labels noise points
Scalability	Good for large data	Can be slow for very large data
Imbalanced sizes	Struggles	Handles well

4.4 Evaluation of Clustering — Silhouette Score

✦ 6-Mark Topic — Formula + numerical

We cannot use accuracy to evaluate clustering (no labels). The **Silhouette Score** measures how well each point fits in its assigned cluster versus how well it would fit in neighbouring clusters.

Silhouette Score Formula

s(i) = (b(i) − a(i)) / max(a(i), b(i))

- **a(i)** = Mean distance from point i to all other points in its OWN cluster (intra-cluster distance). Lower = better.
- **b(i)** = Mean distance from point i to all points in the NEAREST OTHER cluster (inter-cluster distance). Higher = better.

Score Range	Interpretation
s ≈ +1	Point is well inside its cluster — far from neighbouring clusters. Excellent.
s ≈ 0	Point is on the boundary between two clusters.
s ≈ −1	Point is likely in the wrong cluster — closer to another cluster.

NUMERICAL: Silhouette Score Calculation	
Given	Point P is in Cluster 1. Distances from P to other points in Cluster 1: [2, 3, 4] → a(P) = (2+3+4)/3 = 3.0 Distances from P to all points in Cluster 2 (nearest): [5, 6, 7] → b(P) = (5+6+7)/3 = 6.0
Formula	s(P) = (b(P) – a(P)) / max(a(P), b(P)) = (6.0 – 3.0) / max(3.0, 6.0)
Answer	s(P) = 3.0 / 6.0 = 0.50 → Good clustering. P is much closer to its own cluster than to Cluster 2.

4.5 Principal Component Analysis (PCA)

★ 12-Mark Topic — PCA steps + variance explanation + numerical

PCA is a **dimensionality reduction** technique that transforms high-dimensional data into fewer dimensions (principal components) while retaining as much variance (information) as possible. It is used for visualisation, noise reduction, and speeding up ML algorithms.

PCA Steps

- **Step 1 — Standardise:** Subtract mean and divide by std dev for each feature (Z-score scaling). This is mandatory since PCA is variance-based.
- **Step 2 — Covariance Matrix:** Compute the covariance matrix to understand how features vary together. Cov(X) = X■X / (n–1)
- **Step 3 — Eigendecomposition:** Find eigenvalues and eigenvectors of the covariance matrix.
- **Step 4 — Sort:** Sort eigenvalues in descending order. The eigenvector with the largest eigenvalue = first principal component (PC1 — captures most variance).
- **Step 5 — Project:** Multiply original data by the selected top-k eigenvectors to get the reduced dataset.

Explained Variance Ratio

Explained Variance Ratio for PC_i = λ■ / Σλ■

- Choose enough PCs to explain at least 95% of total variance (common threshold).
- Scree plot: Plot eigenvalues vs component number — elbow shows where to cut.

NUMERICAL: PCA — Explained Variance	
Given	4 features with eigenvalues: λ■=3.5, λ■=1.8, λ■=0.5, λ■=0.2 Total variance = 3.5+1.8+0.5+0.2 = 6.0
PC1 EVR	3.5 / 6.0 = 0.583 → 58.3% variance explained
PC2 EVR	1.8 / 6.0 = 0.300 → 30.0% variance explained
PC3 EVR	0.5 / 6.0 = 0.083 → 8.3% variance explained
PC4 EVR	0.2 / 6.0 = 0.033 → 3.3% variance explained
Decision	PC1 + PC2 = 58.3% + 30.0% = 88.3% PC1 + PC2 + PC3 = 88.3% + 8.3% = 96.6% > 95% threshold Keep 3 principal components — reduce from 4D to 3D.

NUMERICAL: PCA — Covariance and Projection (Simple 2D)	
Given	Standardised data matrix X (2 samples, 2 features): x1=[1,–1], x2=[1,–1]
Cov Matrix	Σ = [[1, 1],[1, 1]]
Eigenvalues	Det(Σ – λI) = 0 → λ■ = 2, λ■ = 0
Eigenvector 1	v■ = [1/√2, 1/√2] = [0.707, 0.707] (for λ■=2, PC1)
Projection	Project original data onto PC1: [1,1]·[0.707,0.707] = 1.414 [–1,–1]·[0.707,0.707] = –1.414
Answer	Data reduced from 2D to 1D: [1.414, –1.414] PC1 captures 100% of variance (λ■=2, λ■=0).

4.6 Linear Discriminant Analysis (LDA)

✦ 6-Mark Topic — LDA vs PCA differences — frequent 6-marker

LDA is a **supervised** dimensionality reduction technique. It finds the linear combination of features that best **separates two or more classes** by maximising between-class variance and minimising within-class variance.

LDA Objective

Maximise: J = S_B / S_W

- **S_B** = Between-class scatter matrix (want classes to be far apart)
- **S_W** = Within-class scatter matrix (want each class to be compact)

Aspect	PCA	LDA
Type	Unsupervised	Supervised (uses class labels)
Objective	Maximise total variance	Maximise class separability
Components	Up to min(n_features, n_samples)–1	Up to (n_classes – 1) components
Use Case	General dimensionality reduction	Classification preprocessing
Assumption	No class labels needed	Requires labelled data; assumes normal dist
Output	Principal components	Linear discriminants

Remember: *LDA can also be used as a classifier directly (similar to logistic regression). PCA is purely for dimension reduction.*

4.7 Gaussian Mixture Models (GMM)

✦ 6-Mark Topic — GMM vs K-Means + EM algorithm

GMM is a probabilistic clustering model that assumes the data is generated from a mixture of K Gaussian (normal) distributions, each with its own mean (μ) and covariance (Σ). Unlike K-Means, GMM gives **soft assignments** — each point gets a probability of belonging to each cluster.

GMM Formula

P(x) = Σ π■ × N(x | μ■, Σ■) for k = 1 to K

- π■ = mixing coefficient (weight) of component k. Σπ■ = 1

• N(x | μ■, Σ■) = Gaussian distribution with mean μ■ and covariance Σ■

Expectation-Maximisation (EM) Algorithm

GMM parameters are learned using the **EM algorithm**:

• **E-Step (Expectation):** Compute the probability (responsibility) that each point belongs to each Gaussian component — soft assignment.

• **M-Step (Maximisation):** Update μ, Σ, π for each component using the weighted responsibilities from E-step.

• Repeat E and M steps until convergence (log-likelihood stops improving).

Aspect	K-Means	GMM
Assignment	Hard — each point belongs to exactly one cluster	Soft — each point has probability for each cluster
Cluster Shape	Spherical (circular) — assumes equal variance	Elliptical — flexible covariance
Algorithm	Simple distance minimisation	EM algorithm (probabilistic)
Uncertainty	No — binary membership	Yes — outputs probability distributions
Complexity	Fast, O(nKT)	Slower, more parameters to estimate
Best For	Well-separated, similarly-sized clusters	Overlapping clusters, soft boundaries

QUICK REVISION — Unit 3 & Unit 4

Topic	Key Points / Formula	Marks
Linear Regression	y=β■+β■x, OLS: β=(X■X)■¹X■y, MSE cost, Gradient Descent update rule	12
Logistic Regression	σ(z)=1/(1+e^{-z}), Binary cross-entropy loss, OvR for multiclass	12
Confusion Matrix	TP,FP,TN,FN → Accuracy, Precision, Recall, F1-Score formulae	12
Cross-Validation	k-fold: train on k–1 folds, test on 1. Average k scores.	6
Decision Tree	Entropy = –Σp■log■p■, IG = H(parent)–H(children), Gini = 1–Σp■²	12
KNN	K nearest by Euclidean distance. Majority vote. No training.	6
Naive Bayes	P(C|X) ∝ P(X|C)×P(C). Naive = features independent.	12
SVM	Max margin hyperplane. Support vectors. Kernel trick for non-linear.	12
K-Means	Init→Assign→Update centroids. Elbow method for K. WCSS.	12
Hierarchical	Agglomerative: merge closest. Dendrogram. 4 linkage types.	6
DBSCAN	ε,MinPts. Core/Border/Noise. No K needed. Handles outliers.	6
Silhouette Score	s=(b–a)/max(a,b). Range [–1,+1]. Closer to +1 = better.	6
PCA	Eigenvalues/eigenvectors of cov matrix. EVR=λ■/Σλ. Keep 95% variance.	12
LDA vs PCA	LDA: supervised, maximises class separation. PCA: unsupervised, max variance.	6
GMM	Soft clustering via EM. E-step: responsibilities. M-step: update μ,Σ,π.	6

Textbook: Aurélien Géron — Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow

All the best for your exams! You've got this!